Practical Work №7–10

Data Sorting, Filtering, SQL Queries and Joining Tables

Student: Aruzhan

Group: (write your group in adjust_here.py or on the Excel Settings sheet)

Date: 9 November 2026

Dataset: university educational process from Practice Work 2

1. What was done

I continued the university database from Practice Work 2. It stores students, groups, teachers, departments, courses, classrooms, the class schedule and grades. There are 15 groups, 15 students, 15 teachers, 15 departments, 15 courses, 15 classrooms, 15 schedule rows and 15 grades.

The task sheet uses the names Subjects and grade. In my database the subject table is called Courses, and the mark is stored in grade_value on a scale of 0–100. The queries use those names.

The saved examples use group 1 (IT-21, year 1, Software Engineering), a last-name search for Aliyeva, a search for student id 1001, and a two-condition filter gender = F and group_id <= 5.

To change those examples and rebuild the workbook, the SQL file, the screenshots and this report together, edit the ADJUST block at the top of adjust_here.py and run python3 build.py. To change a filter only in the spreadsheet, use the header arrows on the Students sheet, or the yellow cells on Settings together with the orange Live sheets (Excel 365 or Google Sheets). To change the SQL only, edit the INSERT into settings in sql/practical_work_7_10.sql and run the section “Queries that read the settings table”.

2. Part 1 — Sorting, filtering and searching

I sorted the Students sheet by last name from A to Z. The first row is Abilov Sanzhar and the last row is Zholdas Madina. All 15 students remain visible.

Topic 7. Students sorted by last name

Excel sheet 07_Sort_Last_Name. All students stay in the list; only the order changes.

student_id	last_name	first_name	birth_date	gender	phone	email	group_id
1012	Abilov	Sanzhar	2006-12-16	M	+7-707-2000011	sanzhar.abilov@student.university.example	12
1001	Aliyeva	Aruzhan	2004-01-05	F	+7-707-2000000	aruzhan.aliyeva@student.university.example	1
1014	Amanzhol	Timur	2005-02-18	M	+7-707-2000013	timur.amanzhol@student.university.example	14
1002	Baimukhan	Nursultan	2005-02-06	M	+7-707-2000001	nursultan.baimukhan@student.university.example	2
1009	Beketova	Aigerim	2006-09-13	F	+7-707-2000008	aigerim.beketova@student.university.example	9
1004	Kairatov	Dias	2004-04-08	M	+7-707-2000003	dias.kairatov@student.university.example	4
1013	Kaliyeva	Zarina	2004-01-17	F	+7-707-2000012	zarina.kaliyeva@student.university.example	13
1007	Mukhanova	Dana	2004-07-11	F	+7-707-2000006	dana.mukhanova@student.university.example	7
1010	Ospan	Miras	2004-10-14	M	+7-707-2000009	miras.ospan@student.university.example	10
1015	Serik	Ademi	2006-03-19	F	+7-707-2000014	ademi.serik@student.university.example	15
1003	Suleimenova	Amina	2006-03-07	F	+7-707-2000002	amina.suleimenova@student.university.example	3
1006	Tursyn	Alikhan	2006-06-10	M	+7-707-2000005	alikhan.tursyn@student.university.example	6
1008	Yessen	Islam	2005-08-12	M	+7-707-2000007	islam.yessen@student.university.example	8
1011	Zhakupova	Kamila	2005-11-15	F	+7-707-2000010	kamila.zhakupova@student.university.example	11
1005	Zholdas	Madina	2005-05-09	F	+7-707-2000004	madina.zholdas@student.university.example	5

15 rows

Figure 1. Topic 7. Students sorted by last name.

I also sorted the same sheet by group_id. The order matches the original student_id order, because the Practice Work 2 list places student 1001 in group 1, student 1002 in group 2, and continues in the same way through the last student. The sort is still a group sort: the key is group_id.

Topic 7. Students sorted by group

Excel sheet 07_Sort_Group. Sorted by group_id ascending.

student_id	last_name	first_name	birth_date	gender	phone	email	group_id
1001	Aliyeva	Aruzhan	2004-01-05	F	+7-707-2000000	aruzhan.aliyeva@student.university.example	1
1002	Baimukhan	Nursultan	2005-02-06	M	+7-707-2000001	nursultan.baimukhan@student.university.example	2
1003	Suleimenova	Amina	2006-03-07	F	+7-707-2000002	amina.suleimenova@student.university.example	3
1004	Kairatov	Dias	2004-04-08	M	+7-707-2000003	dias.kairatov@student.university.example	4
1005	Zholdas	Madina	2005-05-09	F	+7-707-2000004	madina.zholdas@student.university.example	5
1006	Tursyn	Alikhan	2006-06-10	M	+7-707-2000005	alikhan.tursyn@student.university.example	6
1007	Mukhanova	Dana	2004-07-11	F	+7-707-2000006	dana.mukhanova@student.university.example	7
1008	Yessen	Islam	2005-08-12	M	+7-707-2000007	islam.yessen@student.university.example	8
1009	Beketova	Aigerim	2006-09-13	F	+7-707-2000008	aigerim.beketova@student.university.example	9
1010	Ospan	Miras	2004-10-14	M	+7-707-2000009	miras.ospan@student.university.example	10
1011	Zhakupova	Kamila	2005-11-15	F	+7-707-2000010	kamila.zhakupova@student.university.example	11
1012	Abilov	Sanzhar	2006-12-16	M	+7-707-2000011	sanzhar.abilov@student.university.example	12
1013	Kaliyeva	Zarina	2004-01-17	F	+7-707-2000012	zarina.kaliyeva@student.university.example	13
1014	Amanzhol	Timur	2005-02-18	M	+7-707-2000013	timur.amanzhol@student.university.example	14
1015	Serik	Ademi	2006-03-19	F	+7-707-2000014	ademi.serik@student.university.example	15

15 rows

Figure 2. Topic 7. Students sorted by group.

I filtered the students to one group, group 1 (IT-21, year 1, Software Engineering). The filter returns Aruzhan Aliyeva.

Topic 7. One selected group

Excel sheet 07_Filter_One_Group. Condition: group_id = 1.

student_id	last_name	first_name	birth_date	gender	phone	email	group_id
1001	Aliyeva	Aruzhan	2004-01-05	F	+7-707-2000000	aruzhan.aliyeva@student.university.example	1

1 row

Figure 3. Topic 7. One selected group.

I searched by student id 1001 and found Aruzhan Aliyeva. I searched by last name Aliyeva and found Aruzhan Aliyeva. In SQL the last-name search uses COLLATE NOCASE, so the capital letters do not matter.

Topic 7. Search by student id

Excel sheet 07_Search. student_id = 1001.

student_id	last_name	first_name	birth_date	gender	phone	email	group_id
1001	Aliyeva	Aruzhan	2004-01-05	F	+7-707-2000000	aruzhan.aliyeva@student.university.example	1

1 row

Figure 4. Topic 7. Search by student id.

Topic 7. Search by last name

Excel sheet 07_Search. last_name = Aliyeva.

student_id	last_name	first_name	birth_date	gender	phone	email	group_id
1001	Aliyeva	Aruzhan	2004-01-05	F	+7-707-2000000	aruzhan.aliyeva@student.university.example	1

1 row

Figure 5. Topic 7. Search by last name.

I then filtered with two conditions at once: gender = F, and group_id <= 5. A row must satisfy both. The result is Aruzhan Aliyeva, Amina Suleimenova, Madina Zholdas. All of these students are in year 1.

Topic 7. Two conditions

Excel sheet 07_Two_Conditions. gender = F AND group_id <= 5.

student_id	last_name	first_name	birth_date	gender	phone	email	group_id
1001	Aliyeva	Aruzhan	2004-01-05	F	+7-707-2000000	aruzhan.aliyeva@student.university.example	1
1003	Suleimenova	Amina	2006-03-07	F	+7-707-2000002	amina.suleimenova@student.university.example	3
1005	Zholdas	Madina	2005-05-09	F	+7-707-2000004	madina.zholdas@student.university.example	5

3 rows

Figure 6. Topic 7. Two conditions.

3. Part 2 — Introduction to SQL

I loaded the eight tables into SQLite (db/university.sqlite) and ran the queries in sql/practical_work_7_10.sql. The first queries only display data. SELECT * FROM Students returns every column. The second query returns student_id, first_name and last_name. SELECT * FROM Groups returns the groups. The subject names come from Courses, because that is the subject table in this database.

Part 2. All records from Students

Every column and every student.

```
SELECT *
FROM Students;
```

student_id	last_name	first_name	birth_date	gender	phone	email	group_id
1001	Aliyeva	Aruzhan	2004-01-05	F	+7-707-2000000	aruzhan.aliyeva@student.university.example	1
1002	Baimukhan	Nursultan	2005-02-06	M	+7-707-2000001	nursultan.baimukhan@student.university.example	2
1003	Suleimenova	Amina	2006-03-07	F	+7-707-2000002	amina.suleimenova@student.university.example	3
1004	Kairatov	Dias	2004-04-08	M	+7-707-2000003	dias.kairatov@student.university.example	4
1005	Zholdas	Madina	2005-05-09	F	+7-707-2000004	madina.zholdas@student.university.example	5
1006	Tursyn	Alikhan	2006-06-10	M	+7-707-2000005	alikhan.tursyn@student.university.example	6
1007	Mukhanova	Dana	2004-07-11	F	+7-707-2000006	dana.mukhanova@student.university.example	7
1008	Yessen	Islam	2005-08-12	M	+7-707-2000007	islam.yessen@student.university.example	8
1009	Beketova	Aigerim	2006-09-13	F	+7-707-2000008	aigerim.beketova@student.university.example	9
1010	Ospan	Miras	2004-10-14	M	+7-707-2000009	miras.ospan@student.university.example	10
1011	Zhakupova	Kamila	2005-11-15	F	+7-707-2000010	kamila.zhakupova@student.university.example	11
1012	Abilov	Sanzhar	2006-12-16	M	+7-707-2000011	sanzhar.abilov@student.university.example	12
1013	Kaliyeva	Zarina	2004-01-17	F	+7-707-2000012	zarina.kaliyeva@student.university.example	13
1014	Amanzhol	Timur	2005-02-18	M	+7-707-2000013	timur.amanzhol@student.university.example	14
1015	Serik	Ademi	2006-03-19	F	+7-707-2000014	ademi.serik@student.university.example	15

15 rows

Figure 7. Part 2. All records from Students.

Part 2. Student id, first name and last name

The task asks for these three columns only.

```
SELECT student_id, first_name, last_name
FROM Students;
```

student_id	first_name	last_name
1001	Aruzhan	Aliyeva
1002	Nursultan	Baimukhan
1003	Amina	Suleimenova
1004	Dias	Kairatov
1005	Madina	Zholdas
1006	Alikhan	Tursyn
1007	Dana	Mukhanova
1008	Islam	Yessen
1009	Aigerim	Beketova
1010	Miras	Ospan
1011	Kamila	Zhakupova
1012	Sanzhar	Abilov
1013	Zarina	Kaliyeva
1014	Timur	Amanzhol
1015	Ademi	Serik

15 rows

Figure 8. Part 2. Student id, first name and last name.

Part 2. All records from Groups

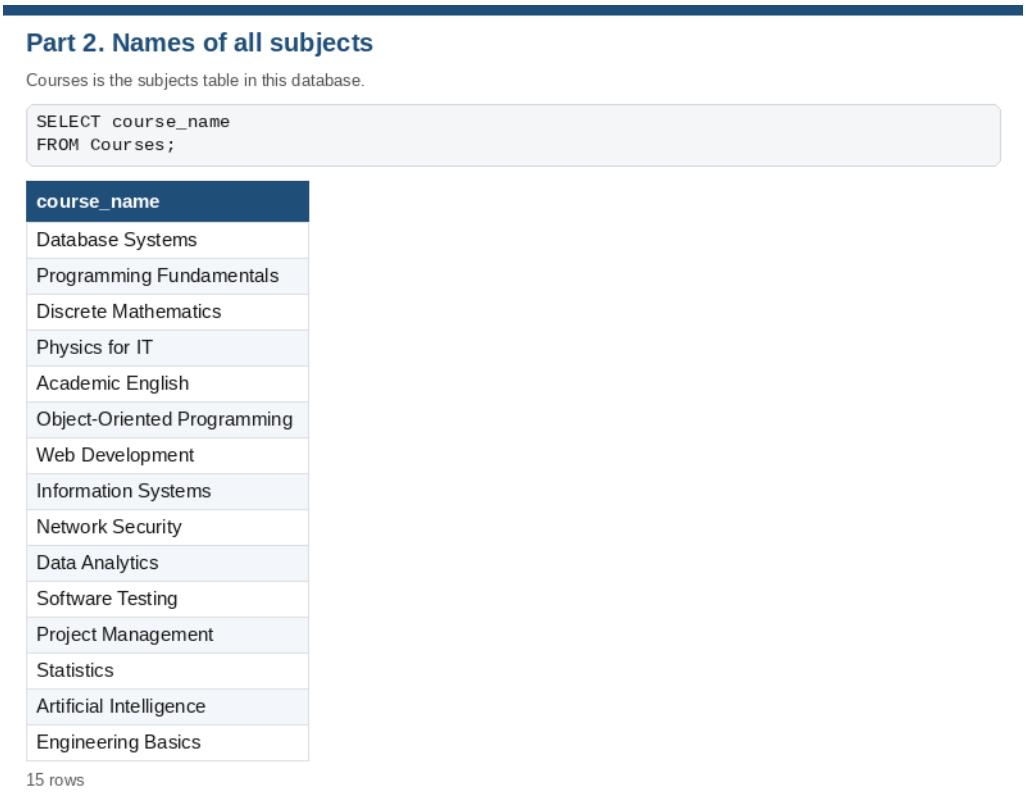
Group number, name, year and specialty.

```
SELECT *
FROM Groups;
```

group_id	group_name	year_of_study	specialty
1	IT-21	1	Software Engineering
2	IT-22	1	Software Engineering
3	IT-23	1	Software Engineering
4	IT-24	1	Software Engineering
5	IT-25	1	Software Engineering
6	IT-26	2	Information Systems
7	IT-27	2	Information Systems
8	IT-28	2	Information Systems
9	IT-29	2	Information Systems
10	IT-30	2	Information Systems
11	IT-31	3	Computer Science
12	IT-32	3	Computer Science
13	IT-33	3	Computer Science
14	IT-34	3	Computer Science
15	IT-35	3	Computer Science

15 rows

Figure 9. Part 2. All records from Groups.



Part 2. Names of all subjects

Courses is the subjects table in this database.

```
SELECT course_name
FROM Courses;
```

course_name
Database Systems
Programming Fundamentals
Discrete Mathematics
Physics for IT
Academic English
Object-Oriented Programming
Web Development
Information Systems
Network Security
Data Analytics
Software Testing
Project Management
Statistics
Artificial Intelligence
Engineering Basics

15 rows

Figure 10. Part 2. Names of all subjects.

4. Part 3 — WHERE, ORDER BY, GROUP BY and aggregates

WHERE group_id = 1 returns Aruzhan Aliyeva. ORDER BY last_name ASC returns all 15 students in alphabetical order. COUNT(*) returns 15. GROUP BY group_id returns one count per group; in the current data, every group has exactly 1 student.

MAX(grade_value) returns 96 (Kamila Zhakupova in Software Testing (2026-05-11)). MIN(grade_value) returns 61 (Alikhan Tursyn in Object-Oriented Programming (2026-05-06)). AVG(grade_value) returns 77.73. The exact SQLite value is 77.73333333333333.

Part 3. Students in the selected group

WHERE keeps only the rows that match.

```
SELECT *
FROM Students
WHERE group_id = 1;
```

student_id	last_name	first_name	birth_date	gender	phone	email	group_id
1001	Aliyeva	Aruzhan	2004-01-05	F	+7-707-2000000	aruzhan.aliyeva@student.university.example	1

1 row

Figure 11. Part 3. Students in the selected group.

Part 3. Students in alphabetical order

ORDER BY sorts the rows. It does not remove any.

```
SELECT *
FROM Students
ORDER BY last_name ASC;
```

student_id	last_name	first_name	birth_date	gender	phone	email	group_id
1012	Abilov	Sanzhar	2006-12-16	M	+7-707-2000011	sanzhar.abilov@student.university.example	12
1001	Aliyeva	Aruzhan	2004-01-05	F	+7-707-2000000	aruzhan.aliyeva@student.university.example	1
1014	Amanzhol	Timur	2005-02-18	M	+7-707-2000013	timur.amanzhol@student.university.example	14
1002	Baimukhan	Nursultan	2005-02-06	M	+7-707-2000001	nursultan.baimukhan@student.university.example	2
1009	Beketova	Aigerim	2006-09-13	F	+7-707-2000008	aigerim.beketova@student.university.example	9
1004	Kairatov	Dias	2004-04-08	M	+7-707-2000003	dias.kairatov@student.university.example	4
1013	Kaliyeva	Zarina	2004-01-17	F	+7-707-2000012	zarina.kaliyeva@student.university.example	13
1007	Mukhanova	Dana	2004-07-11	F	+7-707-2000006	dana.mukhanova@student.university.example	7
1010	Ospan	Miras	2004-10-14	M	+7-707-2000009	miras.ospan@student.university.example	10
1015	Serik	Ademi	2006-03-19	F	+7-707-2000014	ademi.serik@student.university.example	15
1003	Suleimenova	Amina	2006-03-07	F	+7-707-2000002	amina.suleimenova@student.university.example	3
1006	Tursyn	Alikhan	2006-06-10	M	+7-707-2000005	alikhan.tursyn@student.university.example	6
1008	Yessen	Islam	2005-08-12	M	+7-707-2000007	islam.yessen@student.university.example	8
1011	Zhakupova	Kamila	2005-11-15	F	+7-707-2000010	kamila.zhakupova@student.university.example	11
1005	Zholdas	Madina	2005-05-09	F	+7-707-2000004	madina.zholdas@student.university.example	5

15 rows

Figure 12. Part 3. Students in alphabetical order.

Part 3. Total number of students

COUNT(*) counts the rows.

```
SELECT COUNT(*) AS student_count
FROM Students;
```

student_count
15

1 row

Figure 13. Part 3. Total number of students.

Part 3. Number of students in each group

GROUP BY makes COUNT return one row per group.

```
SELECT group_id, COUNT(*) AS student_count
FROM Students
GROUP BY group_id
ORDER BY group_id;
```

group_id	student_count
1	1
2	1
3	1
4	1
5	1
6	1
7	1
8	1
9	1
10	1
11	1
12	1
13	1
14	1
15	1

15 rows

Figure 14. Part 3. Number of students in each group.

Part 3. Maximum grade

The grade column in this database is grade_value.

```
SELECT MAX(grade_value) AS maximum_grade
FROM Grades;
```

maximum_grade
96

1 row

Figure 15. Part 3. Maximum grade.



Figure 16. Part 3. Minimum grade.

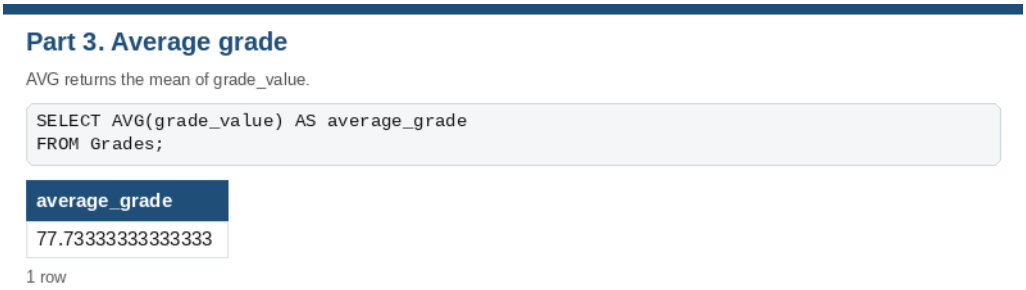


Figure 17. Part 3. Average grade.

5. Part 4 — Joining tables

The tables stay separate. Students stores group_id, not the group name. Groups stores the name once. Grades stores student_id and course_id, not the student's name and not the subject name. JOIN matches the shared id and shows the related columns on one row.

Result	**Column in the first table**	**Column in the second table**
Student name with group name	Students.group_id	Groups.group_id
Student with grade	Grades.student_id	Students.student_id
Subject name with grade	Grades.course_id	Courses.course_id
Department with its courses	Courses.department_id	Departments.department_id

For student 1001, the Students table shows group info 1. After INNER JOIN Groups, the same student shows group info IT-21. The original tables are unchanged. The query only reads them together.

Part 4. Student name and group name

Connected by Students.group_id = Groups.group_id.

```
SELECT
Students.student_id,
Students.first_name,
Students.last_name,
Groups.group_name,
Groups.specialty,
Groups.year_of_study
FROM Students
INNER JOIN Groups
ON Students.group_id = Groups.group_id
ORDER BY Students.last_name, Students.first_name;
```

student_id	first_name	last_name	group_name	specialty	year_of_study
1012	Sanzhar	Abilov	IT-32	Computer Science	3
1001	Aruzhan	Aliyeva	IT-21	Software Engineering	1
1014	Timur	Amanzhol	IT-34	Computer Science	3
1002	Nursultan	Baimukhan	IT-22	Software Engineering	1
1009	Aigerim	Beketova	IT-29	Information Systems	2
1004	Dias	Kairatov	IT-24	Software Engineering	1
1013	Zarina	Kaliyeva	IT-33	Computer Science	3
1007	Dana	Mukhanova	IT-27	Information Systems	2
1010	Miras	Ospan	IT-30	Information Systems	2
1015	Ademi	Serik	IT-35	Computer Science	3
1003	Amina	Suleimenova	IT-23	Software Engineering	1
1006	Alikhan	Tursyn	IT-26	Information Systems	2
1008	Islam	Yessen	IT-28	Information Systems	2
1011	Kamila	Zhakupova	IT-31	Computer Science	3
1005	Madina	Zholdas	IT-25	Software Engineering	1

15 rows

Figure 18. Part 4. Student name and group name.

Part 4. Students and their grades

Connected by Grades.student_id = Students.student_id.

```
SELECT
Students.student_id,
Students.first_name,
Students.last_name,
Grades.course_id,
Grades.grade_value,
Grades.grade_date
FROM Students
INNER JOIN Grades
ON Students.student_id = Grades.student_id
ORDER BY Students.last_name, Students.first_name;
```

student_id	first_name	last_name	course_id	grade_value	grade_date
1012	Sanzhar	Abilov	12	62	2026-05-12
1001	Aruzhan	Aliyeva	1	67	2026-05-01
1014	Timur	Amanzhol	14	76	2026-05-14
1002	Nursultan	Baimukhan	2	74	2026-05-02
1009	Aigerim	Beketova	9	82	2026-05-09
1004	Dias	Kairatov	4	88	2026-05-04
1013	Zarina	Kaliyeva	13	69	2026-05-13
1007	Dana	Mukhanova	7	68	2026-05-07
1010	Miras	Ospan	10	89	2026-05-10
1015	Ademi	Serik	15	83	2026-05-15
1003	Amina	Suleimenova	3	81	2026-05-03
1006	Alikhan	Tursyn	6	61	2026-05-06
1008	Islam	Yessen	8	75	2026-05-08
1011	Kamila	Zhakupova	11	96	2026-05-11
1005	Madina	Zholdas	5	95	2026-05-05

15 rows

Figure 19. Part 4. Students and their grades.

Part 4. Student name, subject and grade

Grades.course_id matches Courses.course_id. Courses stores the subject name.

```
SELECT
Students.first_name,
Students.last_name,
Courses.course_name AS subject,
Grades.grade_value,
Grades.grade_date
FROM Students
INNER JOIN Grades
ON Students.student_id = Grades.student_id
INNER JOIN Courses
ON Grades.course_id = Courses.course_id
ORDER BY Students.last_name, Students.first_name;
```

first_name	last_name	subject	grade_value	grade_date
Sanzhar	Abilov	Project Management	62	2026-05-12
Aruzhan	Aliyeva	Database Systems	67	2026-05-01
Timur	Amanzhol	Artificial Intelligence	76	2026-05-14
Nursultan	Baimukhan	Programming Fundamentals	74	2026-05-02
Aigerim	Beketova	Network Security	82	2026-05-09
Dias	Kairatov	Physics for IT	88	2026-05-04
Zarina	Kaliyeva	Statistics	69	2026-05-13
Dana	Mukhanova	Web Development	68	2026-05-07
Miras	Ospan	Data Analytics	89	2026-05-10
Ademi	Serik	Engineering Basics	83	2026-05-15
Amina	Suleimenova	Discrete Mathematics	81	2026-05-03
Alikhan	Tursyn	Object-Oriented Programming	61	2026-05-06
Islam	Yessen	Information Systems	75	2026-05-08
Kamila	Zhakupova	Software Testing	96	2026-05-11
Madina	Zholdas	Academic English	95	2026-05-05

15 rows

Figure 20. Part 4. Student name, subject and grade.

Part 4. One student before and after the JOIN

The Students table stores the group id. The JOIN adds the group name.

```
SELECT
'Students table' AS source_table,
student_id,
first_name,
last_name,
CAST(group_id AS TEXT) AS group_info
FROM Students
WHERE student_id = 1001
UNION ALL
SELECT
'Students JOIN Groups' AS source_table,
Students.student_id,
Students.first_name,
Students.last_name,
Groups.group_name
FROM Students
INNER JOIN Groups
ON Students.group_id = Groups.group_id
WHERE Students.student_id = 1001;
```

source_table	student_id	first_name	last_name	group_info
Students table	1001	Aruzhan	Aliyeva	1
Students JOIN Groups	1001	Aruzhan	Aliyeva	IT-21

2 rows

Figure 21. Part 4. One student before and after the JOIN.

INNER JOIN returns 15 rows, one for each course. LEFT JOIN returns 17 rows. The extra rows are Economics, General Studies: those departments have no course, so course_name is NULL. Information Technology (2 courses) and Software Engineering (2 courses) appear on more than one row because each course is its own row.

Part 4. INNER JOIN: departments and courses

A department with no course does not appear.

```
SELECT
Departments.department_name,
Courses.course_name
FROM Departments
INNER JOIN Courses
ON Departments.department_id = Courses.department_id
ORDER BY Departments.department_id, Courses.course_name;
```

department_name	course_name
Computer Science	Database Systems
Information Technology	Programming Fundamentals
Information Technology	Web Development
Mathematics	Discrete Mathematics
Physics	Physics for IT
Languages	Academic English
Software Engineering	Object-Oriented Programming
Software Engineering	Software Testing
Information Systems	Information Systems
Cybersecurity	Network Security
Data Science	Data Analytics
Engineering	Engineering Basics
Management	Project Management
Statistics	Statistics
Artificial Intelligence	Artificial Intelligence

15 rows

Figure 22. Part 4. INNER JOIN: departments and courses.

Part 4. LEFT JOIN: every department

Every department stays. A missing course is NULL.

```
SELECT
Departments.department_name,
Courses.course_name
FROM Departments
LEFT JOIN Courses
ON Departments.department_id = Courses.department_id
ORDER BY Departments.department_id, Courses.course_name;
```

department_name	course_name
Computer Science	Database Systems
Information Technology	Programming Fundamentals
Information Technology	Web Development
Mathematics	Discrete Mathematics
Physics	Physics for IT
Languages	Academic English
Software Engineering	Object-Oriented Programming
Software Engineering	Software Testing
Information Systems	Information Systems
Cybersecurity	Network Security
Data Science	Data Analytics
Engineering	Engineering Basics
Economics	*NULL*
Management	Project Management
Statistics	Statistics
Artificial Intelligence	Artificial Intelligence
General Studies	*NULL*

17 rows

Figure 23. Part 4. LEFT JOIN: every department.

6. Individual assignment

1. What is the difference between sorting and filtering?

Sorting changes the order of the rows and keeps every row. Filtering hides the rows that do not match a condition. Searching finds a row by a specific value, such as a student id or a last name. In this work, sorting Students by last name still shows all 15 students, with Abilov first. Filtering to group 1 (IT-21, year 1, Software Engineering) shows Aruzhan Aliyeva and hides the other students.

2. What is SQL used for?

SQL is the language used to work with a relational database. It creates tables, inserts the rows, and then retrieves or updates them. I used it to display the university tables, to filter and sort students, to summarise grades with COUNT, MAX, MIN and AVG, and to bring related tables together with JOIN.

3. What is the difference between WHERE and ORDER BY?

WHERE decides which rows are included. ORDER BY decides the order of the rows that were included. WHERE group_id = 1 returns only that group. ORDER BY last_name ASC returns every student, arranged alphabetically. WHERE does the filtering. ORDER BY does the sorting. One query can use both.

4. Why do we use GROUP BY and aggregate functions?

An aggregate function turns many rows into one value. COUNT counts rows, MAX and MIN find the largest and smallest values, and AVG calculates the mean. GROUP BY divides the rows into groups before that calculation, so the function returns one result per group. COUNT(*) on Students returns 15. With GROUP BY group_id, the same function returns a separate count for each group (every group has exactly 1 student). On Grades, MAX is 96, MIN is 61 and the average is 77.73.

5. What is the purpose of JOIN?

JOIN combines columns from related tables into one result. The database stores each fact once: the group name lives in Groups, and Students stores only group_id. Matching Students.group_id to Groups.group_id shows the student's name and the group name together without copying the group name into every student row.

6. What is the difference between INNER JOIN and LEFT JOIN?

INNER JOIN returns only rows that have a match in both tables. LEFT JOIN returns every row from the left table, and fills the right-hand columns with NULL when there is no match. INNER JOIN returns 15 rows, one for each course. LEFT JOIN returns 17 rows. The extra rows are Economics, General Studies: those departments have no course, so course_name is NULL.

7. Conclusion

The same university tables can be sorted and filtered in the spreadsheet and queried in SQLite. WHERE and the Excel filter choose rows, ORDER BY and the Excel sort choose the order, GROUP BY with aggregate functions summarises rows, and JOIN reads related tables together through the shared ids.

Files: University_Database.xlsx, db/university.sqlite, sql/practical_work_7_10.sql, sql/query_results.txt, the screenshots folder, and this report.